

1. Title Page

Project Title: Bus Ticket Reservation System

Subject: Software Engineering

Group Number: 1

Group Members:

- **Name:** Om Patel
- **Name:** Soham Patel

Submitted to: [Your Professor's Name] **Date of Submission:** [Date]

2. Problem Statement

The current methods for bus ticket booking are largely manual, leading to significant inefficiencies and frustration for both travelers and bus operators. Passengers face a time-consuming and often confusing process to find bus schedules, confirm seat availability, and secure reservations, especially during peak holiday seasons. The lack of a centralized system can result in overbookings, errors in passenger data, and a cumbersome process for ticket cancellations.

For bus operators, the manual management of bookings is labor-intensive and error-prone. It is difficult to get real-time insights into seat availability, manage different fare classes, and generate reports on a specific route's performance. The absence of a digital record-keeping system makes data analysis and revenue management challenging.

This project aims to solve these problems by developing a **Bus Ticket Reservation System**, a robust and reliable application that automates and streamlines the entire booking process. The system will provide an intuitive interface for users to effortlessly search, book, and manage their tickets, while simultaneously offering bus operators an efficient platform for managing their routes and bookings. The ultimate goal is to create a seamless, error-free, and transparent experience for all stakeholders.

3. SDLC Model

For this project, the **Evolutionary Process Model** has been selected. This model is based on the iterative development of a system prototype, which is ideal for a user-facing application like a ticket reservation system. It allows the development team to build a core set of features and refine them based on feedback before moving on to the next set.

The phases of this model will be applied as follows:

1. **Communication:** The initial requirements for a core feature, such as the bus search functionality, are gathered and discussed. The team decides on the key search parameters and the format for displaying results.
2. **Quick Plan & Modeling:** A high-level plan is created, and a quick design is sketched out. For instance, the design for the bus search result page will be planned to show bus numbers, routes, departure times, and available seats clearly.
3. **Construction of Prototype:** A working, albeit basic, prototype of the bus search feature is coded. This prototype will perform a search and display results, but may not yet be

connected to a live booking system.

4. **Deployment, Delivery & Feedback:** The prototype is presented to stakeholders (or a sample group of users) to gather feedback. This feedback loop is crucial for refining the user experience. For example, if users find the seat selection process confusing, the team can go back and redesign it immediately.

This iterative approach mitigates the risk of a late-stage design failure and ensures that the final product is highly user-centric, intuitive, and robust.

4. Requirements (Functional & Non-Functional)

Functional Requirements

- **FR-1: User Authentication & Profile Management:**
 - FR-1.1: Users must be able to securely register with a unique username and password.
 - FR-1.2: Users must be able to log in to their account.
 - FR-1.3: Users can view and update their profile information.
- **FR-2: Bus Search & Discovery:**
 - FR-2.1: The system must allow users to search for available buses based on origin, destination, and departure date.
 - FR-2.2: Search results shall display bus schedules, including departure/arrival times, travel duration, and fare.
 - FR-2.3: The system must display the number of available seats for each bus in the search results.
- **FR-3: Ticket Reservation:**
 - FR-3.1: Users must be able to view an interactive seat map of a bus.
 - FR-3.2: Users can select one or more available seats.
 - FR-3.3: The system shall calculate the total fare based on the number of selected seats.
 - FR-3.4: A user can confirm their booking, and the selected seats will be marked as reserved.
- **FR-4: Booking Management:**
 - FR-4.1: Users can view a history of all their past and upcoming bookings.
 - FR-4.2: Users can cancel a reservation, and the seat will be released.
- **FR-5: Administrator Functions:**
 - FR-5.1: An administrator can securely log in to an admin dashboard.
 - FR-5.2: The administrator can add, update, and delete bus routes and schedules.
 - FR-5.3: The administrator can view comprehensive booking reports, including passenger lists and revenue.

Non-Functional Requirements

- **NFR-1: Performance:**
 - NFR-1.1: All search queries must return results within **3-5 seconds**.

- NFR-1.2: The ticket booking transaction must be completed within **10 seconds** to prevent timeouts.
- **NFR-2: Security:**
 - NFR-2.1: All passwords must be stored using a one-way cryptographic hash function.
 - NFR-2.2: The administrator's access must be protected by strong authentication.
- **NFR-3: Usability:**
 - NFR-3.1: The Command-Line Interface (CLI) must be intuitive, with clear prompts and navigation.
 - NFR-3.2: The display of schedules and seat maps must be easy to read and understand.
- **NFR-4: Reliability:**
 - NFR-4.1: The system must operate without crashing during normal use.
 - NFR-4.2: The data stored in the database must be consistent and free from corruption.

5. UML Diagrams

To visually represent the system, the following UML diagrams are essential.

- **Use Case Diagram:** This diagram will identify the key actors in the system (Traveler, Admin) and their interactions. It will provide a high-level overview of the system's functionality. For example, the Traveler can Search for Buses, Book a Ticket, and View Booking History. The Admin can Manage Routes and View Reports.
- **Class Diagram:** This diagram will illustrate the static structure of the system, showing the classes, their attributes, and their relationships.
 - **User Class:** Attributes like user_id, name, email, password.
 - **Bus Class:** Attributes like bus_id, bus_number, total_seats, seats_booked.
 - **Route Class:** Attributes like route_id, source, destination, fare.
 - **Reservation Class:** Attributes like reservation_id, user_id, bus_id, seat_number.
 - The relationships would show that a User can have many Reservations, a Bus has one Route, and a Reservation is linked to both a User and a Bus.
- **Sequence Diagram:** This diagram will model the dynamic behavior of a specific interaction. For the **"Book a Ticket"** use case, it would show the sequence of messages between objects:
 1. The User object sends a search_buses(origin, destination, date) message to the System object.
 2. The System object queries the Database object.
 3. The Database returns a list of available Bus objects.
 4. The System displays the list to the User.
 5. The User selects a bus and sends a select_seat(bus_id, seat_number) message to the System.
 6. The System updates the Bus object to mark the seat as reserved.
 7. The System creates a new Reservation object and saves it to the Database.
 8. The System sends a confirmation message to the User.

6. System Design

The system will be designed with a three-tier architecture, ensuring that the user interface, business logic, and data storage are separate and modular.

- **Presentation Layer (Client-Side):** This layer is the user interface. It will be a simple, text-based **Command-Line Interface (CLI)** built with Python. This choice makes the application lightweight and easy to deploy on various platforms. It will use a clean, logical design to guide the user through the booking process.
- **Business Logic Layer (Application Layer):** This is the core of the system. It will be implemented as a set of Python scripts and modules. This layer handles all the processing, including:
 - Validating user input.
 - Managing search and filtering logic.
 - Handling booking requests and seat availability.
 - Interacting with the data layer.
- **Data Layer (Database):** This layer will be a local, file-based database managed by **SQLite**. SQLite is a powerful, serverless database that is perfect for this project's scale. It will store all the system data, including User accounts, Bus schedules, Routes, and Reservations. The use of SQL commands will ensure data integrity and reliable data retrieval.

7. Testing Plan

A comprehensive testing plan is critical to ensuring the system is reliable and meets all requirements.

- **Unit Testing:** We will test individual functions and methods to ensure they perform their specific tasks correctly.
 - **Test Case 7.1.1:** Test login() with correct credentials. **Expected Result:** Login successful.
 - **Test Case 7.1.2:** Test login() with incorrect credentials. **Expected Result:** "Invalid username or password" message is displayed.
 - **Test Case 7.1.3:** Test is_seat_available() for a booked seat. **Expected Result:** False is returned.
- **Integration Testing:** We will test the interaction between different modules.
 - **Test Case 7.2.1:** Test the end-to-end booking process. **Steps:** Search for a bus, select a seat, confirm the booking. **Expected Result:** A reservation record is created in the database, and the seat is marked as unavailable on the bus record.
 - **Test Case 7.2.2:** Test the cancellation process. **Steps:** Cancel a booked ticket. **Expected Result:** The reservation record is deleted, and the seat is marked as available.
- **System Testing:** We will test the entire application as a complete system to ensure all functional and non-functional requirements are met.
 - **Test Case 7.3.1: Performance Test:** Measure the time it takes to perform a bus search query with a large dataset (e.g., 1000 routes). **Expected Result:** The query returns results within 5 seconds.

- **Test Case 7.3.2: Security Test:** Attempt to access admin features without authentication. **Expected Result:** Access is denied, and a login prompt is displayed.

8. Screenshots of Python Output

[Placeholder for screenshots of the running Python application. Insert images showcasing the main menu, bus search results, a visual representation of the seat map, the booking confirmation message, and a screenshot of the admin's booking report.]

9. Conclusion & Future Scope

This **Bus Ticket Reservation System** provides a foundational solution for automating the bus booking process. It successfully fulfills all the functional and non-functional requirements, offering a simple yet effective tool for both travelers and administrators. The system is designed to be reliable, secure, and user-friendly, providing a significant improvement over manual booking methods.

The project can be expanded in the future with several enhancements:

- **Graphical User Interface (GUI):** A more modern and visually appealing GUI can be developed using libraries like Tkinter, PyQt, or a web-based framework.
- **Online Payment Gateway:** Integration with a payment service (e.g., Stripe, PayPal) would enable users to pay for their tickets directly through the application.
- **Real-time Updates:** Implementing a system for real-time seat availability updates could prevent overbookings and improve the user experience.
- **Mobile Application:** The system could be developed into a native Android or iOS application to reach a wider user base.
- **Dynamic Fare Pricing:** An algorithm could be implemented to adjust ticket prices based on demand, time of day, or booking volume.